

Trabajo fin de grado

Desarrollo de una aplicación web para la valoración y gestión de listas de videojuegos



Wilson Javier Simbaña Ganazhapa

Escuela Politécnica Superior
Universidad Autónoma de Madrid
C/ Francisco Tomás y Valiente nº 11

**UNIVERSIDAD AUTÓNOMA DE MADRID
ESCUELA POLITÉCNICA SUPERIOR**



Grado en Ingeniería Informática

TRABAJO FIN DE GRADO

**Desarrollo de una aplicación web para la
valoración y gestión de listas de videojuegos**

**Autor: Wilson Javier Simbaña Ganazhapa
Tutor: Germán Montoro Manrique**

junio 2025

Todos los derechos reservados.

Queda prohibida, salvo excepción prevista en la Ley, cualquier forma de reproducción, distribución comunicación pública y transformación de esta obra sin contar con la autorización de los titulares de la propiedad intelectual.

La infracción de los derechos mencionados puede ser constitutiva de delito contra la propiedad intelectual (*arts. 270 y sgts. del Código Penal*).

DERECHOS RESERVADOS

© por UNIVERSIDAD AUTÓNOMA DE MADRID

Francisco Tomás y Valiente, n.º 1

Madrid, 28049

Spain

Wilson Javier Simbaña Ganazhapa

Desarrollo de una aplicación web para la valoración y gestión de listas de videojuegos

Wilson Javier Simbaña Ganazhapa

C\ Francisco Tomás y Valiente N.º 11

IMPRESO EN ESPAÑA – PRINTED IN SPAIN

AGRADECIMIENTOS

En primer lugar, me gustaría agradecer a mi tutor German Montoro por su apoyo, por permitirme realizar este proyecto y ayudarme a realizar esta memoria.

Por otro lado, también quiero agradecer a mis compañeros Pachi, Alba, Simón y Perucho quienes me han acompañado en toda la carrera y con quienes he vivido momentos inolvidables, tardes de sufrimiento en los laboratorios de la EPS y en la biblioteca y momentos divertidos en la terraza de la cafetería. Gracias a ellos, me he convertido en la persona que soy ahora.

Por último, no me quiero olvidar de mi familia. A mis padres que estuvieron apoyándome en todo momento, incluso cuando tuve dudas de dejar la carrera. También a mi hermano quien me convenció a seguir en esta carrera, a no rendirme y, sobre todo, a demostrarme a mi mismo que sería capaz de superar mis límites.

RESUMEN

Hoy en día, Internet es una gran fuente de información que nos ayuda a tomar decisiones, muchas veces basadas en las opiniones de otros usuarios. En el sector del entretenimiento, las valoraciones y reseñas juegan un papel fundamental, influyendo en la elección de películas, música o videojuegos. Como resultado, han surgido múltiples plataformas que permiten a los usuarios compartir sus opiniones e interactuar con otras personas.

Sin embargo, en el ámbito específico de los videojuegos, existen pocas plataformas que ofrezcan una experiencia centrada en la valoración personal, la organización de listas personalizadas y la interacción social entre jugadores. Por ello, se plantea el desarrollo de una aplicación web que permita a los usuarios valorar videojuegos, organizarlos en listas y compartir su actividad con otros usuarios, cubriendo así una necesidad poco atendida en este sector.

PALABRAS CLAVE

WishList, PlayedList, Valoración, Amistad, Actividad, Listas personalizadas, Videojuegos

ABSTRACT

Nowadays, the Internet is a major source of information that helps us make decisions, often based on the opinions of other users. In the entertainment sector, ratings and reviews play a key role, influencing the choice of movies, music, or video games. As a result, multiple platforms have emerged that allow users to share their opinions and interact with others.

However, in the specific field of video games, there are few platforms that offer an experience focused on personal ratings, customized game lists, and social interaction between players. Therefore, this project proposes the development of a web application that enables users to rate video games, organize them into lists, and share their activity with other users, addressing a need that is still largely unmet in this sector.

KEYWORDS

WishList, PlayedList, Rating, Friendship, Activity, Custom Lists, Videogames

ÍNDICE

1	Introducción	1
1.1	Motivación	1
1.2	Objetivo	1
1.3	Organización de la memoria	1
2	Estado del arte	3
2.1	Actualidad	3
2.1.1	Plataformas	3
2.2	Frameworks	4
2.2.1	Frameworks frontend	5
2.2.2	Frameworks backend	5
3	Diseño	7
3.1	Análisis de requisitos	7
3.1.1	Requisitos funcionales	7
3.1.2	Requisitos no funcionales	8
3.2	Diagramas	8
3.2.1	Casos de uso	8
3.2.2	Diagramas de flujo	11
3.2.3	Diagrama de base de datos	12
3.3	Mockups	13
4	Implementación	21
4.1	Arquitectura del sistema	21
4.2	Comunicación frontend - backend	21
4.3	Subsistemas y navegación	23
4.3.1	Navegación entre pantallas	23
4.3.2	Subsistema de autenticación	24
4.3.3	Subsistema de búsqueda	25
4.3.4	Subsistema de valoración	27
4.3.5	Subsistema de listas	28
4.3.6	Subsistema de amistad	30
4.4	Pruebas	34
5	Conclusión	37

5.1 Conclusiones	37
5.2 Trabajo futuro	37
Bibliografía	39

LISTAS

Lista de códigos

4.1	Protección urls	22
4.2	Pruebas django	35

Lista de figuras

3.1	Diagrama de caso de uso	11
3.2	Flujo realizar valoración	11
3.3	Flujo crear lista personalizada	12
3.4	Flujo editar lista personalizada	13
3.5	Diagrama de base de datos	14
3.6	Vista principal	15
3.7	Vista principal con filtros	15
3.8	Información del videojuego	16
3.9	Listas personalizadas	17
3.10	Creación de listas personalizadas	17
3.11	Amistad entre usuarios	18
3.12	Solicitudes de amistad pendientes	18
3.13	Actividad de otros usuarios	19
4.1	Mapeo de respuesta	22
4.2	Respuesta del servidor	23
4.3	Intento de acceso a una ruta protegida	23
4.4	Navegación entre pantallas	24
4.5	Prueba PostMan	25
4.6	Coincidencias en desarrolladora	26
4.7	Componentes implicados para la pantalla de detalle de videojuego	26
4.8	Componentes implicados para la pantalla de inicio	27
4.9	Diálogo de valoración	28
4.10	Mensaje de confirmación de valoración	29
4.11	Valoraciones de un usuario	29
4.12	Componentes implicados para la sección 'Mis listas'	31

4.13 Componentes implicados para la sección 'Amistades'	32
4.14 Componentes implicados para la sección 'Amistades'	33
4.15 Salida TestCase	34
4.16 Pruebas Cypress comprobación de elementos	36
4.17 Pruebas Cypress interacción del usuario	36

INTRODUCCIÓN

1.1. Motivación

El desarrollo de este trabajo surge con el objetivo de añadir funcionalidades que no están presentes en otras plataformas existentes. Hasta ahora, los usuarios solo podían valorar videojuegos, almacenarlos en listas personalizadas y buscarlos por género, título o año de lanzamiento. Sin embargo, estas aplicaciones no permiten una interacción real entre usuarios, lo que limita su potencial como red social centrada en los videojuegos.

Además, algunas necesidades específicas, como la búsqueda de videojuegos según la desarrolladora, no están contempladas en estas plataformas. Por todo ello, este proyecto propone el desarrollo de una aplicación web que permita una mayor interacción entre usuarios y amplíe las posibilidades de búsqueda, ofreciendo así una experiencia más completa y personalizada.

1.2. Objetivo

El objetivo principal de este trabajo es desarrollar una aplicación web que permita a los usuarios valorar videojuegos, crear listas personalizadas y almacenar títulos en dichas listas. Asimismo, la aplicación ofrecerá la posibilidad de visualizar la actividad de otros usuarios, fomentando así la interacción social dentro de la plataforma.

Además, se ampliarán las opciones de búsqueda de videojuegos, incorporando un nuevo filtro de búsqueda por desarrolladora de videojuegos, con el fin de que los usuarios puedan descubrir y valorar títulos desarrollados por sus estudios favoritos.

1.3. Organización de la memoria

La memoria se organiza en cuatro capítulos sin incluir el capítulo de introducción:

En el segundo capítulo, se explica el estado de arte donde se comparan plataformas actuales y se indican las funcionalidades se han añadido en este trabajo. Además, se describen las tecnologías actuales utilizadas en el desarrollo.

En el tercer capítulo, se presenta la descripción del proyecto, detallando los requisitos establecidos, así como diagramas y maquetas que describen el diseño de la aplicación.

En el cuarto capítulo, se explica la arquitectura del sistema y la comunicación entre los distintos componentes de la aplicación, junto a una descripción de las tecnologías empleadas. También, se incluye una explicación sobre las pruebas realizadas en distintos módulos.

En el quinto capítulo, se definen las conclusiones de este trabajo, indicando el trabajo futuro y posibles mejoras para mejorar la aplicación.

ESTADO DEL ARTE

2.1. Actualidad

Actualmente, existen plataformas web que permiten valorar videojuegos, pero la mayoría de ellas carecen de funciones de interacción entre usuarios o de filtros de búsqueda avanzados y accesibles. Estas carencias afectan negativamente a la experiencia del usuario, ya que limitan la posibilidad de consultar referencias de otros jugadores o de encontrar sus títulos favoritos de forma más eficiente.

2.1.1. Plataformas

Para la realización de este trabajo de fin de grado, se han revisado las principales plataformas de valoración y gestión de videojuegos. En general, uno de los aspectos más mejorables detectados en estas plataformas es el sistema de búsqueda y la accesibilidad a los filtros, elementos clave para una navegación eficiente.

RAWG

RAWG es una plataforma web de videojuegos que permite a los usuarios descubrir títulos y consultarlos según su plataforma o popularidad. Además, ofrece información detallada sobre cada videojuego, como capturas de pantalla, valoraciones de usuarios, enlaces a páginas web donde pueden adquirirse y las consolas en las que están disponibles. También cuenta con una API que permite a los desarrolladores acceder a estos datos para integrarlos en sus propias aplicaciones.

Sin embargo, RAWG resenta limitaciones en su sistema de búsqueda, ya que no permite aplicar varios filtros de forma simultánea. Si un usuario desea filtrar los videojuegos simultáneamente por género y por año de lanzamiento, no puede hacerlo de forma directa. En su lugar, debe realizar búsquedas secuenciales: primero seleccionar el género y, posteriormente, filtrar por año.

IGDB

IGDB (Internet Game Database) es una base de datos de videojuegos lanzada por Twitch en 2014 [1] que almacena información sobre miles de títulos. Una de sus características destacadas es un contador para los videojuegos más esperados, que muestra los días, horas y minutos restantes para su lanzamiento. Además de permitir valorar videojuegos, IGDB ofrece una funcionalidad adicional: los usuarios pueden valorar los comentarios de otros mediante un sistema de 'me gusta' o 'no me gusta', de forma similar a los comentarios en YouTube.

En cuanto al sistema de búsqueda, presenta una mejora en este aspecto, ya que permite aplicar múltiples filtros en una sola búsqueda. Sin embargo, la accesibilidad a estos filtros no es óptima, pues el usuario debe primero hacer clic en la barra de búsqueda y luego seleccionar "Advanced search" para que se desplieguen las opciones de filtrado.

Backloggd

Backloggd es una plataforma web que ofrece funcionalidades similares a las de RAWG e IGDB, como la valoración y organización de videojuegos. Una de sus características más destacadas es la posibilidad de responder a los comentarios de otros usuarios, lo que fomenta la interacción y el debate en torno a aspectos positivos y negativos de los videojuegos. Además, cuando un título forma parte de una saga o trilogía, la plataforma muestra los demás juegos relacionados, facilitando al usuario el seguimiento completo de la serie.

En cuanto a los filtros, Backloggd presenta un flujo de navegación más complejo. Para acceder a los filtros, el usuario debe ir a la sección "Games", hacer clic en "Apply Filters" y, a continuación, seleccionar los criterios deseados, lo que añade pasos adicionales que podrían simplificarse.

2.2. Frameworks

Un framework es un conjunto de herramientas que facilita el desarrollo de aplicaciones. Proporciona una base sólida y reutilizable para crear aplicaciones más rápidas, robustas y eficientes.

Para el desarrollo de este proyecto se han valorado diferentes frameworks con el fin de buscar un desarrollo eficiente y seguro. Antes de analizar las opciones disponibles, es importante destacar que una aplicación web consta de dos partes fundamentales [2]:

Backend Se encarga de la lógica, la gestión de datos y la conexión a la base de datos. Esta parte no es visible para el usuario final

Frontend Es la interfaz con la que interactúa el usuario. Incluye elementos como texto, botones e imágenes con el fin de facilitar su navegación.

2.2.1. Frameworks frontend

Vue.js

Vue.js [3] es un framework desarrollado en JavaScript. Su arquitectura está basado en componentes, lo que significa que cada elemento de la interfaz encapsula su lógica propia y estilo permitiendo una reutilización del código en distintas partes de la aplicación. La curva de aprendizaje es baja, lo cual facilita el desarrollo para aplicaciones sencillas. Además, cuenta con herramientas integradas que simplifican la interacción con formularios y el manejo del estado.

React

React [4] es una biblioteca de Javascript basado en componentes. Utiliza JSX, una sintaxis que combina HTML con Javascript facilitando la creación de componentes reutilizados. Cuenta un alto rendimiento al actualizar elementos de la interfaz. A diferencia del resto de frameworks, React se enfoca exclusivamente en la interfaz del usuario, por lo que requiere incorporar librería adicionales para funcionalidades como el enrutamiento o la gestión de formularios.

Angular

Angular [5] es un framework desarrollado en TypeScript basado en componentes. A diferencia de Vue.js y React, Angular destaca por una estructura más rígida al proyecto, lo que facilita el mantenimiento en aplicaciones grandes. El uso de TypeScript, proporciona un tipado estático, por lo que mejora la detección de errores durante el desarrollo. Además, ofrece un sistema de comunicación entre componentes y dispone de un sistema de inyección de dependencias. Sin embargo, su curva de aprendizaje es más alta respecto a las otras dos.

Tras comparar los frameworks frontend, se ha optado por Angular debido a su tipado estático proporcionado por TypeScript y su robusto sistema de comunicación e inyección de dependencias entre componentes, características que garantizan mayor fiabilidad y escalabilidad en el desarrollo de la aplicación.

2.2.2. Frameworks backend

Django

Django [6] es un framework web escrito en Python. Sus principales características son su fácil curva de aprendizaje, el sistema de plantillas para las vistas, un panel de administración incorporado y un ORM (Object-Relational Mapping) integrado. Esta última característica permite interactuar con bases de datos relacionales utilizando objetos del lenguaje sin necesidad de escribir sentencias SQL.

Además, incluye por defecto protección contra ataques CSRF, lo que mejora la seguridad de las aplicaciones desarrolladas.

.NET

.NET [7] es un framework desarrollado por Microsoft. Su característica más destacada es el CLR (Common Language Runtime), que permite integrar proyectos escritos en diferentes lenguajes compatibles como C#, F# y Visual Basic. También incluye un ORM (Entity Framework) para la gestión de bases de datos. A diferencia de Django, no ofrece un panel de administración preconfigurado, por lo que es necesario desarrollarlo manualmente. Además, aunque su configuración inicial es más avanzada, requiere mayor conocimiento técnico.

Spring boot

Spring Boot [8] es un framework basado en Java que simplifica la creación de aplicaciones web al reducir considerablemente la configuración inicial. Esto permite a los desarrolladores crear aplicaciones listas para producción con cambios mínimos. Si bien no incluye un panel de administración por defecto, su ecosistema es muy amplio y permite una alta personalización. También es compatible con herramientas como Hibernate o JPA para la gestión de bases de datos mediante ORM.

Una vez comparados los frameworks backend disponibles, se ha optado por Django debido a su sencilla curva de aprendizaje y a la inclusión de un panel de administración por defecto. Esta funcionalidad resulta especialmente útil en este proyecto, ya que permite gestionar fácilmente los videojuegos almacenados en la base de datos desde una interfaz gráfica sin necesidad de desarrollar una.

DISEÑO

3.1. Análisis de requisitos

Durante la fase de diseño del software, la definición de los requisitos es fundamental, ya que proporciona una descripción completa del comportamiento y funcionamiento del sistema. Por ello, es necesario establecerlos previamente de manera clara y estructurada.

3.1.1. Requisitos funcionales

Los requisitos funcionales son los siguientes:

RF-1.— Subsistema de autenticación

RF-1.1.— El sistema permite a los usuarios registrarse mediante correo electrónico, nombre de usuario y contraseña.

RF-1.1.1.— El usuario deberá escribir un correo electrónico válido y nombre de usuario único y confirmar la contraseña.

RF-1.2.— El sistema permite a los usuarios identificarse mediante usuario y contraseña.

RF-1.3.— El sistema no permite registrar a usuarios con el mismo correo o nombre de usuario.

RF-2.— Subsistema de valoración

RF-2.1.— El sistema permite a los usuarios realizar valoraciones de los videojuegos, las cuales quedarán asociadas a su cuenta.

RF-2.1.1.— El usuario debe seleccionar una calificación entre 1 y 5 estrellas, donde 1 representa la puntuación más baja y 5 la más alta.

RF-2.1.2.— Opcionalmente, el usuario podrá escribir un comentario.

RF-3.— Subsistema de listas de videojuegos

RF-3.1.— El sistema permite a los usuarios añadir videojuegos a una lista de deseos (wishlist) para marcar aquellos que quieren jugar.

RF-3.2.— El sistema permite a los usuarios eliminar videojuegos de su lista de deseos (wishlist) cuando ya no desean jugarlos.

RF-3.3.— El sistema permite a los usuarios marcar los videojuegos que han jugado.

RF-3.4.— El sistema permite a los usuarios desmarcar los videojuegos que han jugado, previamente

indicados como jugados.

RF-3.5.– El sistema permite a los usuarios crear listas de videojuegos personalizadas ya sean jugados o no.

RF-3.5.1.– Deberá indicar, el nombre de la lista y los videojuegos asociados a esa lista.

RF-3.6.– El sistema permite a los usuarios eliminar listas personalizadas de videojuegos.

RF-3.7.– El sistema permite eliminar videojuegos que están en las listas personalizadas del usuario.

RF-4.– Subsistema de amistad

RF-4.1.– El sistema permite a los usuarios enviar solicitudes de amistad a otros usuarios.

RF-4.1.1.– La solicitud deberá incluir el nombre de usuario del destinatario.

RF-4.2.– El sistema indica al usuario si tiene solicitudes pendientes.

RF-4.2.1.– El usuario podrá aceptar o rechazar dichas solicitudes.

RF-4.3.– El sistema permite al usuario visualizar la actividad de los usuarios con los que ha establecido amistad.

RF-4.3.1.– Se mostrará la lista de videojuegos valorados, junto con su puntuación y comentario (si lo hubiera).

RF-5.– Subsistema de búsqueda

RF-5.1.– El sistema permite a los usuarios buscar videojuegos por título, género, año de lanzamiento y/o desarrolladora.

RF-5.2.– El sistema muestra información relevante de cada videojuego, como el año de lanzamiento, la desarrolladora, la valoración media y los comentarios de los usuarios.

3.1.2. Requisitos no funcionales

RNF-1.– El sistema debe responder a las acciones del usuario en un tiempo máximo de 2 segundos

RNF-2.– El sistema debe proteger la información de los usuarios y evitar accesos no autorizados.

RNF-3.– Las contraseñas son almacenadas de forma segura pasando por un proceso de hashing.

3.2. Diagramas

3.2.1. Casos de uso

A continuación se detallan los casos de uso:

CU-001

Nombre de caso de uso: Creación de lista personalizada.

Actores involucrados: Usuario, base de datos(externo).

Resumen: El usuario crea una lista personalizada de videojuegos.

Pre-condiciones: Ninguna.

Post-condiciones: La lista queda registrada en el sistema y visible para el usuario.

Curso básico de eventos:

- 1.– El usuario introduce su nombre de usuario, correo electrónico y contraseña en el formulario de registro.
- 2.– El sistema redirige al usuario a un formulario de autenticación.
- 3.– El sistema introduce sus nombre de usuario y contraseña en el formulario de autenticación.
- 4.– El sistema redirige al usuario a la página principal.
- 5.– El usuario accede a la sección de lista personalizadas.
- 6.– El usuario selecciona la opción 'Crear lista'.
- 7.– El usuario indica el nombre de la lista y agrega los videojuegos que desea incluir.
- 8.– El usuario confirma la operación.
- 9.– El sistema guarda la lista, muestra un mensaje de confirmación y la muestra en la sección de lista personalizadas.

Caminos alternativos:

- 1.– El usuario introduce un nombre de usuario o un correo electrónico existente y/o contraseña inválida en el formulario de registro. El sistema muestra un mensaje de error indicando que vuelva a rellenar el formulario de registro.
- 2.– El usuario introduce un nombre de usuario y/o contraseña inválidos en el formulario de autenticación. El sistema muestra un mensaje de error indicando que vuelva a rellenar el formulario de autenticación.
- 3.– Si el usuario no indica nombre y/o no agrega videojuegos, el sistema notifica al usuario con un mensaje de error.
- 4.– El usuario cancela la operación. El sistema no crea la lista y redirige al usuario a la sección de listas.
- 5.– El usuario cierra sesión antes de completar la operación. El sistema no crea la lista y redirige al usuario al formulario de autenticación.

CU-002

Nombre de caso de uso: Valoración de un videojuego.

Actores involucrados: Usuario, base de datos(externo).

Resumen: El usuario realiza una valoración de un videojuegos asignando una puntuación y un comentario opcional.

Pre-condiciones: El usuario se ha registrado e identificado en el sistema.

Post-condiciones: La valoración se guarda en el sistema y queda reflejado en la información del videojuego.

Curso básico de eventos:

- 1.– El usuario selecciona el videojuego que quiere valorar.
- 2.– El usuario selecciona la opción de 'Valorar'.
- 3.– El usuario indica la puntuación entre 1-5 estrellas y, opcionalmente, un comentario.

- 4.– El usuario confirma la valoración.
- 5.– El sistema almacena la valoración y actualiza la valoración media del videojuego.
- 6.– El sistema muestra un mensaje de confirmación y muestra la valoración creada.

Caminos alternativos:

- 1.– El usuario no introduce la puntuación. El sistema muestra un mensaje de error y solicita que complete el campo.
- 2.– El usuario cancela la operación. El sistema no almacena la valoración y le redirige a la información del videojuego.

CU-003

Nombre de caso de uso: Crear amistad.

Actores involucrados: Usuario, base de datos(externo).

Resumen: El usuario envía una solicitud de amistad a otro usuario.

Pre-condiciones: El usuario se ha registrado e identificado en el sistema.

Post-condiciones: Se crea la amistad si el otro usuario acepta la solicitud.

Curso básico de eventos:

- 1.– El usuario accede a la sección de amistad.
- 2.– El usuario selecciona la opción de 'Añadir amigo'.
- 3.– El usuario introduce el nombre de usuario con el que quiere establecer una amistad.
- 4.– El sistema envía la solicitud al otro usuario y muestra un mensaje de confirmación de que la solicitud ha sido enviada.
- 5.– El otro usuario accede a la sección de amistad.
- 6.– El otro usuario selecciona la opción de 'Solicitudes pendientes'.
- 7.– El otro usuario acepta la solicitud de amistad.
- 8.– El sistema muestra un mensaje de confirmación y actualiza la lista de amistades.

Caminos alternativos:

- 1.– El usuario introduce un nombre de usuario no válido, el sistema no muestra nada.
- 2.– El usuario vuelve a enviar una solicitud de amistad al mismo usuario. El sistema muestra un mensaje de error indicando que ya existe una solicitud pendiente o una amistad..
- 3.– El otro usuario rechaza la solicitud. El sistema no crea amistad y no actualiza la lista de amistades.

Por último, se muestra un único diagrama de casos de uso. Esta decisión permite mostrar una visión general y simplificada del sistema, evitando la fragmentación en múltiples diagramas. Además, ayuda a eliminar redundancias en los actores y casos de uso que podrían repertirse si se dividiera en varios.

En la figura 3.1, se recogen todas las funcionalidades que puede realizar el usuario dentro del sistema.

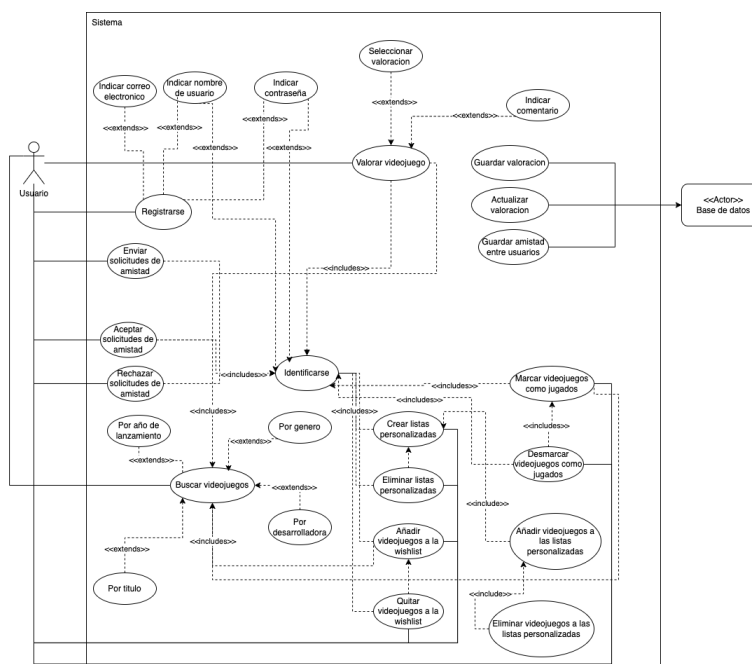


Figura 3.1: En este diagrama se muestran las acciones que realiza el usuario en la aplicación

3.2.2. Diagramas de flujo

Realizar valoración

En la figura 3.2, se observa el comportamiento del sistema cuando el usuario realiza la valoración del videojuego. Una vez que el usuario se autentica, este podrá ver la información del videojuego y, a continuación, realizar la valoración. Para llevar a cabo la valoración, debe rellenar el formulario. Una vez que el usuario envía el formulario, el backend valida la información y responde 'OK' que indica que la operación ha sido exitosa.

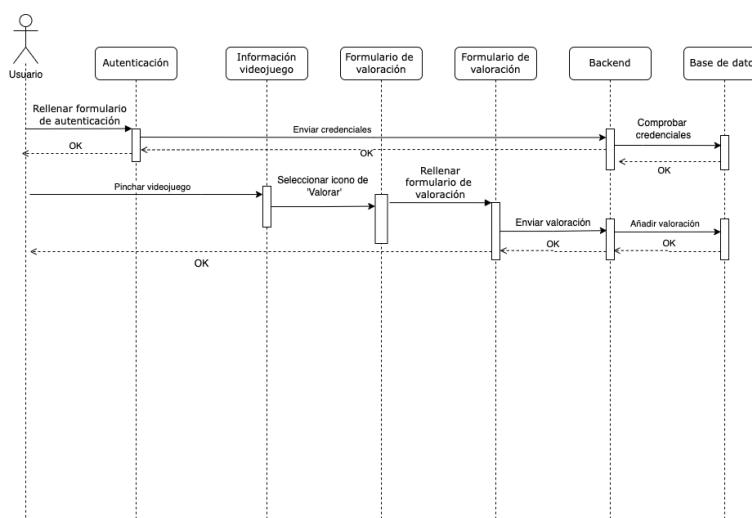


Figura 3.2: En este diagrama se puede ver el proceso completo de la valoración de un videojuego

Crear lista

En la figura 3.2, se observa el comportamiento del sistema cuando el usuario crea una lista personalizada. Tras autenticarse, el usuario accede a la sección de 'Mis listas' y selecciona la opción de 'Crear lista'. A continuación, rellena el formulario con la información adecuada que es enviada al backend, quien almacena la nueva lista y devuelve 'OK' que indica que la operación ha sido exitosa.

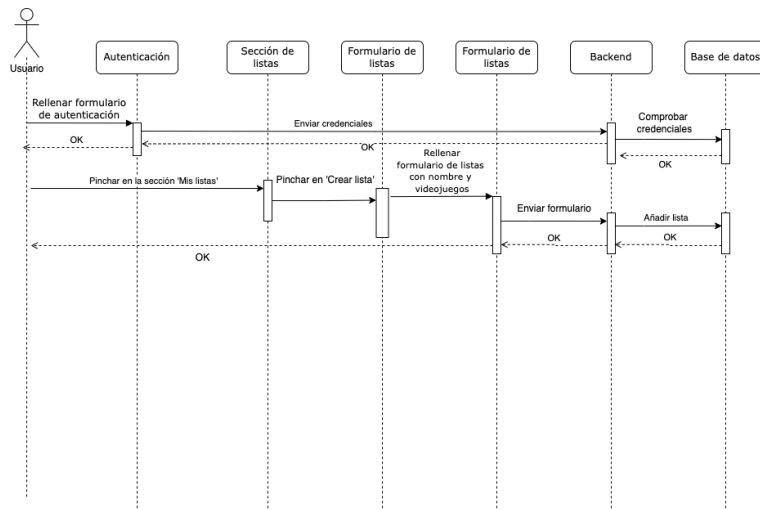


Figura 3.3: En este diagrama se puede ver el proceso completo de la creación de la lista personalizada

Editar lista

Tras completar el paso anterior, el usuario puede editar su lista. El usuario selecciona la lista que desea actualizar. A continuación, rellena el formulario con los datos nuevos. Esta información es enviada al backend, que actualiza el contenido y responde 'OK'. Este proceso se ilustra en la figura 3.4

3.2.3. Diagrama de base de datos

La base de datos de la aplicación contiene las siguientes tablas:

Usuarios: Almacena la información de los usuarios como el correo electrónico, nombre de usuario y contraseña.

Videojuegos: Contiene la información sobre los videojuegos disponibles. Incluye campos como el título, año de lanzamiento, género, resumen, desarrolladora e imagen. El campo "género" puede tomar valores como: acción, deporte, simulación, estrategia, arcade, musicales, juegos de mesa y aventura. La imagen se representa mediante una URL para evitar almacenar archivos multimedia dentro del proyecto.

Amistad: Almacena las amistades entre los usuarios. En esta tabla se almacena los iden-

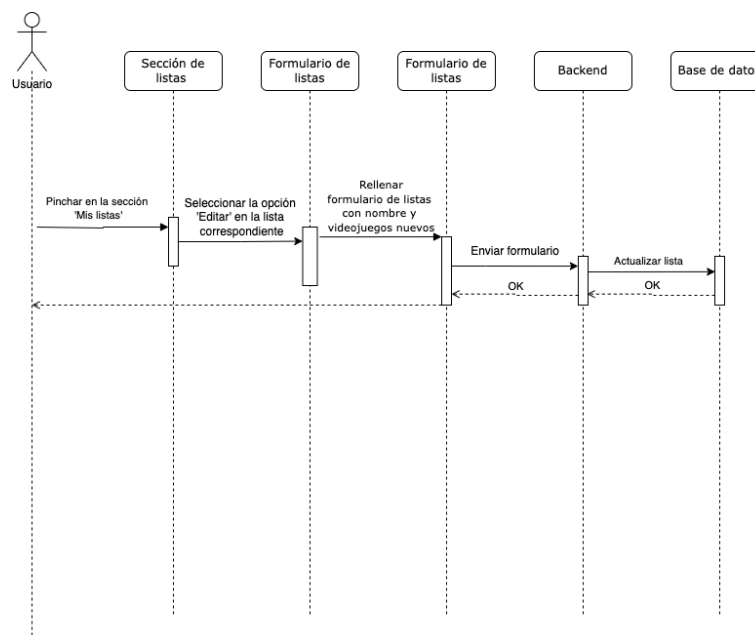


Figura 3.4: En este diagrama se puede ver el proceso completo de la edición de la lista personalizada

tificadores de los usuarios involucrados y el estado que puede ser: aceptado, rechazado o pendiente.

Wishlist: Almacena los videojuegos que el usuario desea jugar en un futuro. Como su comportamiento es similar a la lista de videojuegos jugados (playedList), se establece el campo 'wishlist' para diferenciarlos.

Valoraciones: Almacena las valoraciones que realizan los usuario sobre los videojuegos.

Listas personalizadas: Almacena las listas personalizadas de los usuarios para organizar sus videojuegos.

Elementos listas: Esta tabla asocia cada videojuego a una lista correspondiente.

En la figura 3.5 se muestra el diseño de la base de datos, cada tabla con sus atributos y claves correspondientes.

3.3. Mockups

Los mockups son representaciones visuales de la interfaz de usuario de un proyecto que anticipar y evaluar el diseño final antes de su implementación. Su objetivo es ofrecer una visión clara de los elementos gráficos, de la estructura y la disposición de los componentes de la aplicación.

Para el desarrollo de los mockups, se ha utilizado la herramienta Figma. A continuación se detallan las principales vistas que se han realizado.

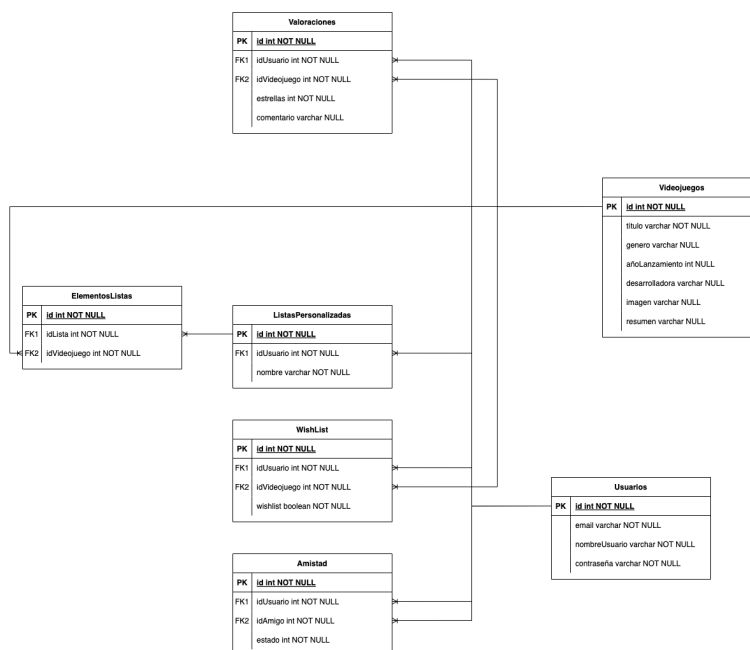


Figura 3.5: En este diagrama se muestran los campos de todas las tablas relacionales

Pantalla principal

En la figura 3.6 se muestra la pantalla principal de la aplicación a la que accede el usuario tras autenticarse. En la parte superior se encuentra una barra de navegación que permite navegar entre las distintas secciones de la aplicación. Justo debajo, se encuentra una barra de búsqueda acompañada un boton desplegable con distintos filtros como se observa figura 3.7. Por último, se presenta un listado de videojuegos mostrados mediante sus imágenes, con soporte para paginación.

Opciones videojuegos

En la figura 3.8 se muestra la vista con la información detallada del videojuego, a la que accede al usuario tras hacer clic en su imagen. En esta pantalla se presentan distintas opciones como añadir el videojuego a la wishlist, marcarlo como jugado (played list) o realizar una valoración. Estas acciones están disponibles mediante iconos situados a la derecha de la imagen del videojuego. En orden descendente, las acciones asociadas a los iconos son: añadir o eliminar videojuegos de la wishlist, añadir o eliminar videojuegos de la played list y realizar una valoración. A la izquierda de la imagen, cuenta con un botón para volver a la página anterior en la que se encontraba el usuario.

Listas personalizadas

En la figura 3.9 se ilustra la vista correspondiente a las listas personalizadas del usuario. En esta pantalla, el usuario puede crear una nueva lista indicando un nombre y los videojuegos que quiere asociar, tal y como se muestra en la figura 3.10. Además, puede consultar el contenido de cada lista,

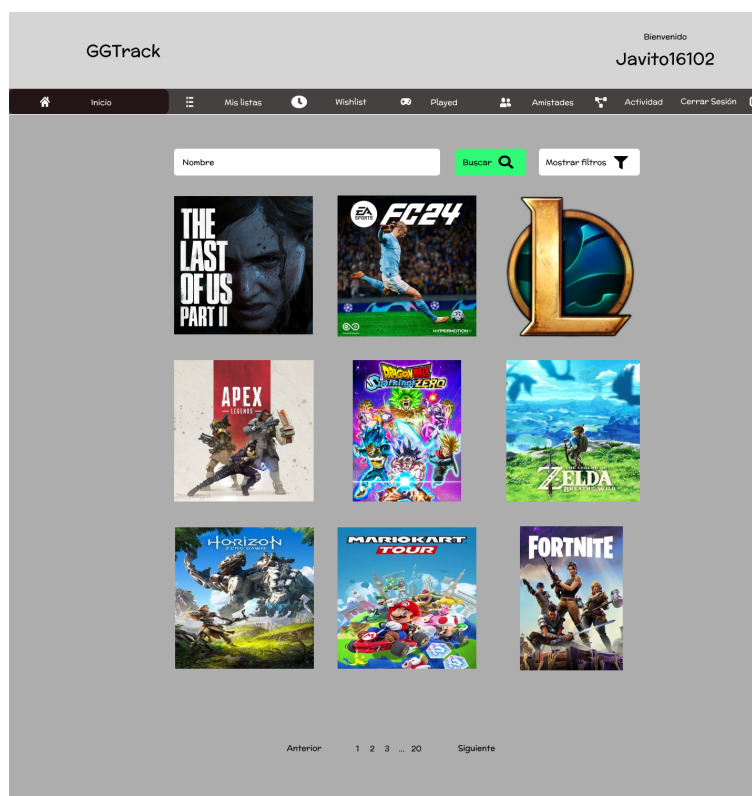


Figura 3.6: Vista principal de la aplicación

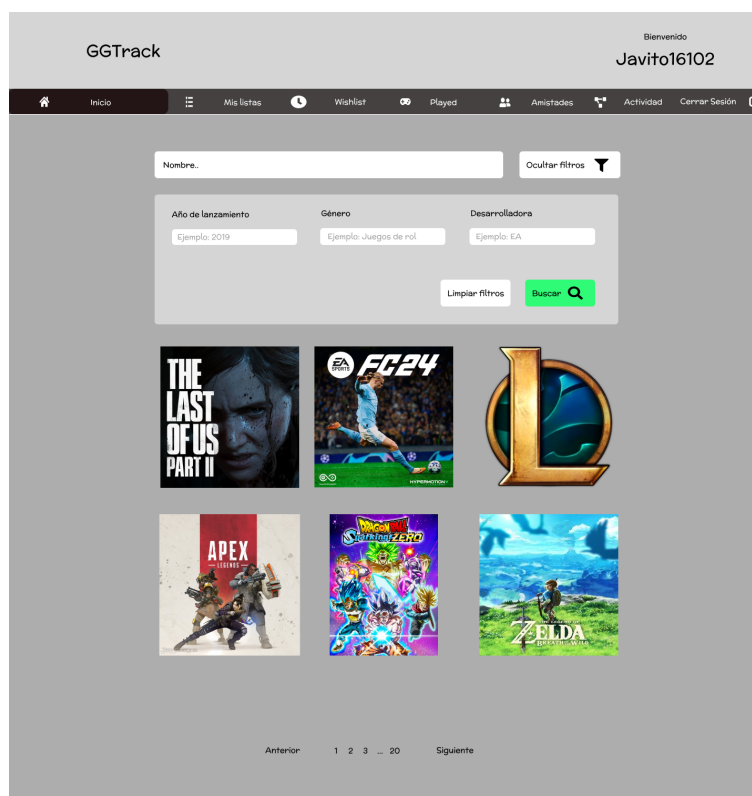


Figura 3.7: Vista principal de la aplicación con filtros desplegados

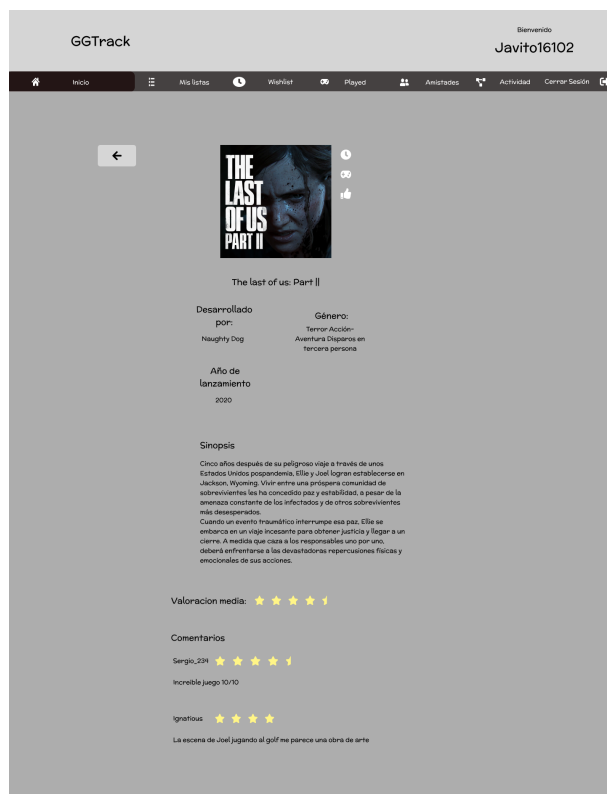


Figura 3.8: Vista donde se muestra la información del videojuego

así como eliminarla o editarla.

Amistad

En la figura 3.11 se muestra la pantalla correspondiente a las amistades del usuario. En esta vista se divide en dos secciones principales: una en la que se listan los usuarios con los que ha establecido amistad, y otra en la que se muestran las solicitudes de amistad pendientes, como se ilustra en la figura 3.12

Actividad

Por último, la figura 3.13 se muestra la pantalla donde se visualiza la actividad de otros usuarios. Como se observa, se incluye las valoraciones que ha realizado dicho usuario incluyendo la puntuación y el comentario, si existe.

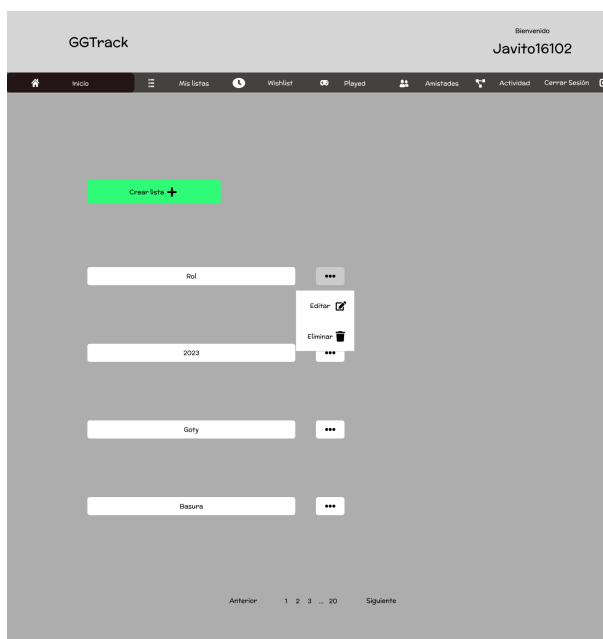


Figura 3.9: Vista donde se muestran las listas personalizadas y las diferentes acciones disponibles como eliminar y editar

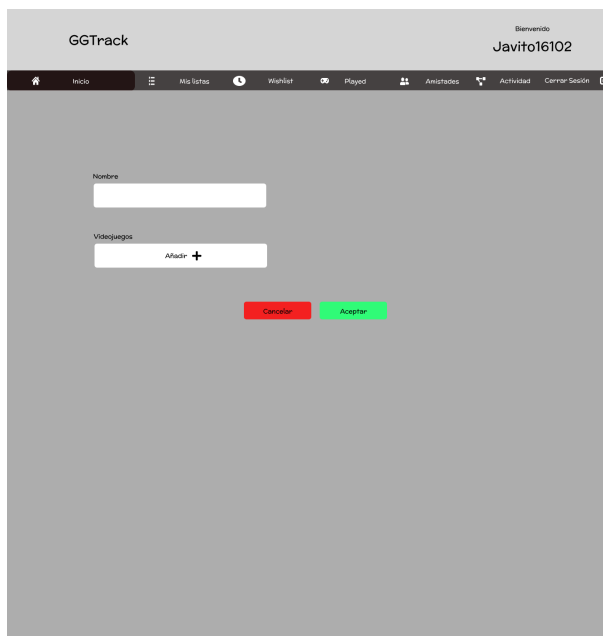


Figura 3.10: Vista donde se muestra un formulario para la creación de vistas personalizadas

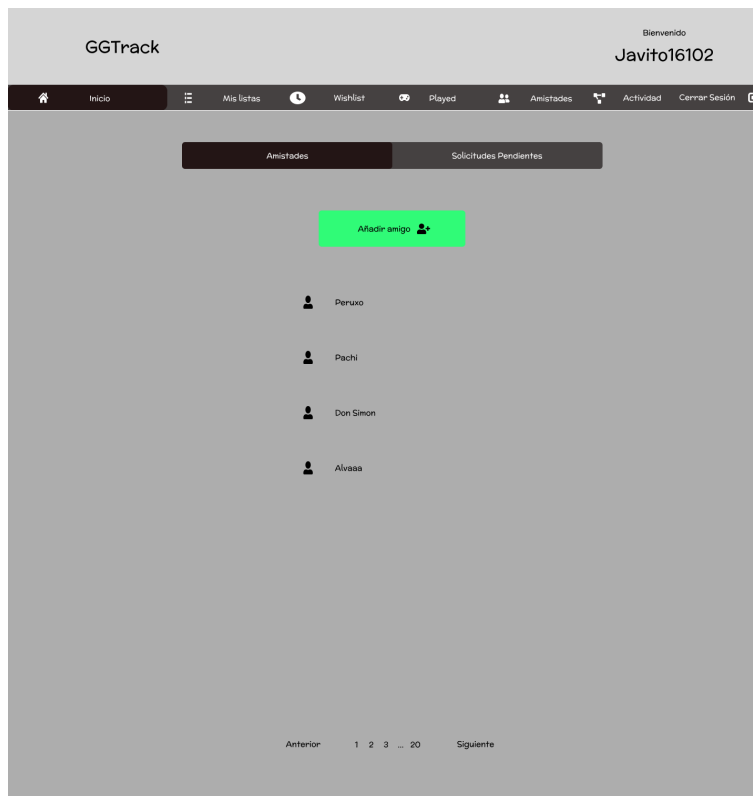


Figura 3.11: Vista donde se muestra los usuarios con lo que tiene amistad

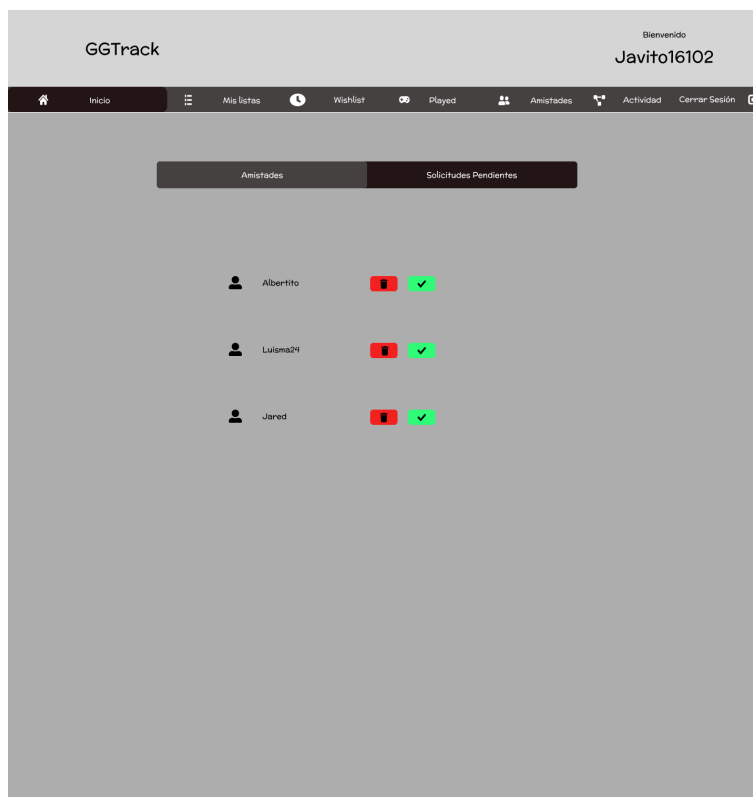


Figura 3.12: Vista donde se muestran las solicitudes de amistad pendientes y las diferentes acciones disponibles como aceptar y eliminar

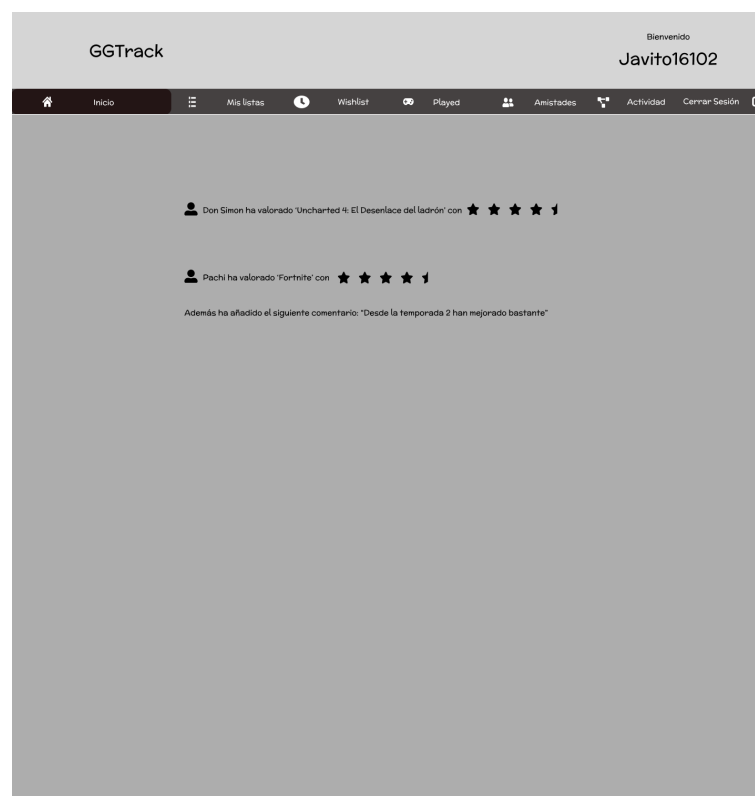


Figura 3.13: Vista donde se muestra la actividad de otros usuarios

IMPLEMENTACIÓN

4.1. Arquitectura del sistema

La aplicación sigue una arquitectura **cliente-servidor** [9] dividida en **tres capas**:

Capa de presentación: desarrollada en Angular. En esta capa, se encuentran los componentes que permiten al usuario interactuar con la aplicación.

Capa de lógica de negocio: gestionada por Django. Se han implementado viewsets por cada funcionalidad de la aplicación.

Capa de datos: los datos se almacenan en una base de datos de PostgreSQL, que es gestionada por el ORM de Django.

Esta arquitectura permite modificar una capa sin necesidad de afectar directamente a las demás. Además, tanto backend como base de datos se pueden reutilizar en otras aplicaciones.

4.2. Comunicación frontend - backend

La comunicación entre frontend y backend se realiza mediante **API REST**

Frontend

Por el lado del frontend, gracias al módulo HttpClient de Angular, se realizan peticiones HTTP al servidor. Este módulo se utiliza mediante la inyección de dependencias, lo que facilita su uso en los componentes de la aplicación. Con el uso de este módulo, se han implementado tres servicios principales que se encargan de gestionar las operaciones relacionadas con el backend:

AuthService: se encarga de enviar las peticiones relacionadas con la autenticación, registro y cierre de sesión de los usuarios.

SearchService: encargado de las peticiones GET.

UpdateService: encargado de las peticiones relacionadas con la actualización o eliminación de los recursos del servidor.

Una vez que el servidor responde, se transforman los datos en objetos comprensibles por Angular mediante funciones de mapeo. Dicha transformación permite modificar los datos para adaptarlos a las necesidades de la aplicación, como por ejemplo, mostrar el genero del videojuego de forma más legible en la aplicación o aplicar funciones de filtrado sobre los objetos transformados.

En la figura 4.1, se observa como se realiza el mapeo de los datos a objetos de Angular.

```
private mapResponseToVideogame(data: any): Videojuegos {  
  return {  
    id: data.id,  
    titulo: data.título,  
    resumen: data.resumen,  
    imagen: data.imagen,  
    desarrolladora: data.desarrolladora,  
    genero: data.genero,  
    añoLanzamiento: data.añoLanzamiento,  
  };  
}
```

Figura 4.1: En esta figura se muestra el mapeo de los datos al objeto Videojuegos

Backend

Por el lado del backend, las rutas de la API están protegidas mediante la librería Djoser que valida automáticamente la presencia y validez del token JWT. En la figura 4.1 se muestra cómo se protegen las rutas de la aplicación.

Código 4.1: En esta figura se presenta la protección de las rutas usando Djoser

```
1 from django.contrib import admin  
2 from django.urls import path, include  
3 from apiWeb import api  
4  
5 urlpatterns = [  
6     path('admin/', admin.site.urls),  
7     path('api/v1/', include('djoser.urls')),  
8     path('api/v1/', include('djoser.urls.authtoken')),
```

Si el token, no está presente o no es válido, se devuelve un error 401. En caso contrario, cada viewset ejecuta la lógica correspondiente, implicando operaciones de búsquedas, actualizaciones o inserciones en la base de datos mediante el ORM de Django. Por último, la respuesta se envía al frontend en formato JSON, junto con el código de respuesta HTTP correspondiente. En la figura 4.2 se muestra un ejemplo de respuesta del servidor al frontend.



Figura 4.2: En esta figura se muestra la lista de videojuegos devuelta por el servidor

4.3. Subsistemas y navegación

A continuación, se describen los subsistemas implementados en la aplicación y su funcionamiento, tanto en el frontend como en el backend. También se explica la navegación entre las distintas pantallas y donde se encuentra cada subsistema.

4.3.1. Navegación entre pantallas

La navegación entre pantallas se realiza mediante Angular Router, permitiendo elegir a cada pantalla un componente específico. Además cada pantalla se protege mediante un guard, para comprobar si el usuario se ha autenticado en el sistema. En caso de que quiera acceder a una ruta específica sin haber iniciado sesión, se redirige a la pantalla de inicio de sesión.

En la figura 4.3, en la ventana de inspección, se observa que se ha realizado la petición a la pantalla de 'Mis-listas', pero el guard ha redirigido al usuario a la pantalla de inicio de sesión.

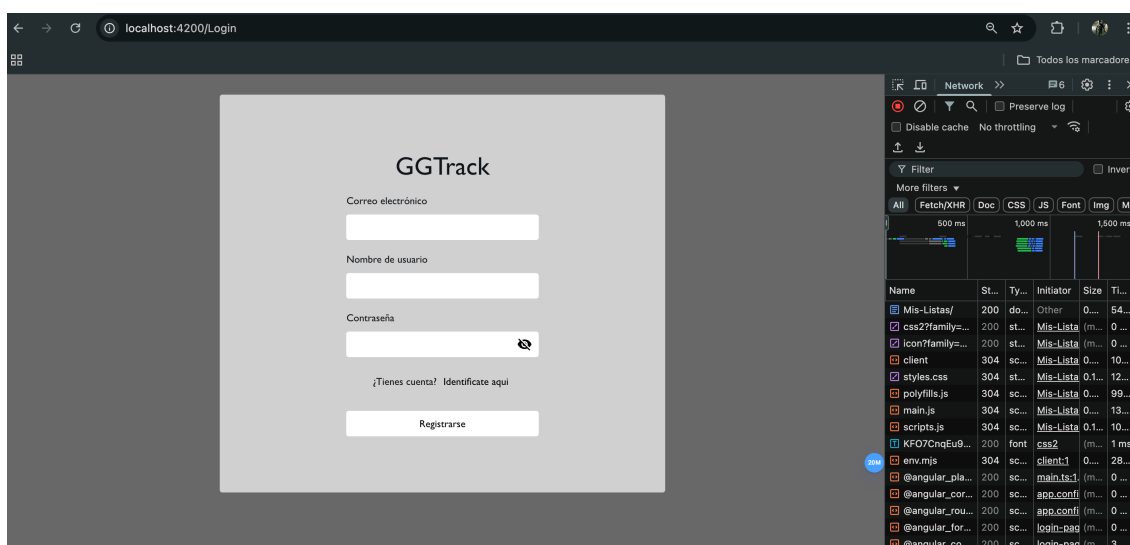


Figura 4.3: Intento de acceso a una ruta protegida

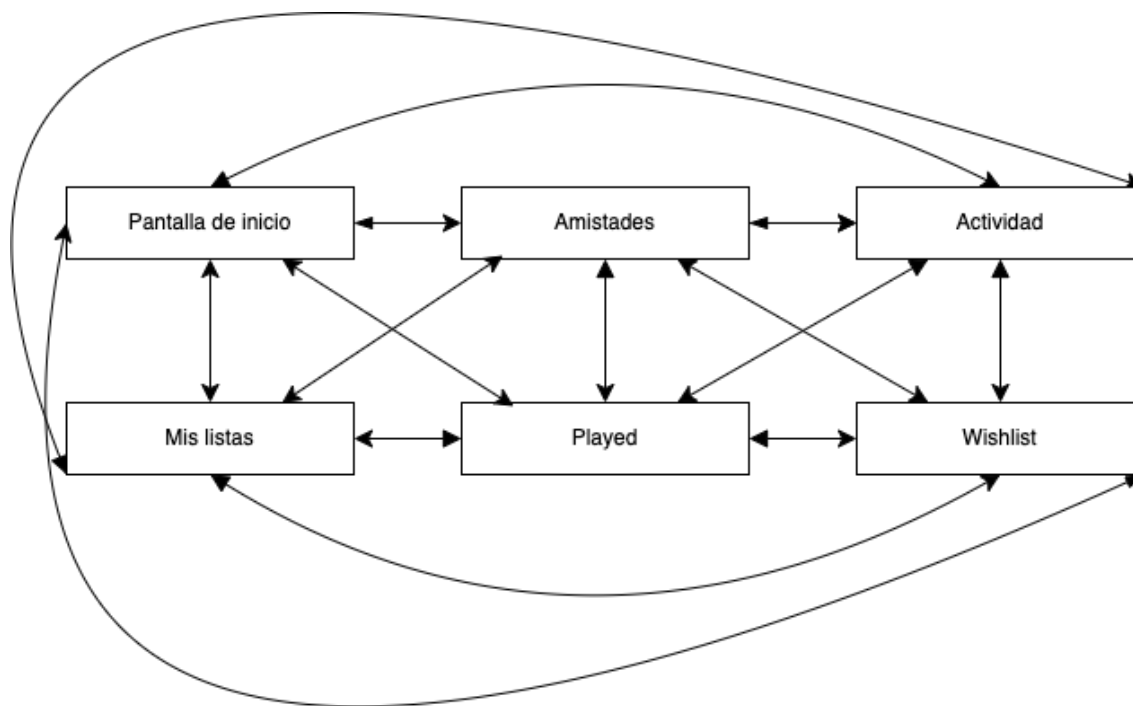


Figura 4.4: Navegación entre pantallas

En la figura 4.4 se muestra el esquema de navegación entre las distintas pantallas de la aplicación.

4.3.2. Subsistema de autenticación

El subsistema de autenticación, accesible desde la pantalla de inicio de sesión, permite a los usuarios registrarse e iniciar sesión y obtener un token JWT para realizar las peticiones al backend.

Por el lado del frontend, el componente LoginComponent contiene los formularios de registro y acceso, que se implementan mediante Angular Reactive Forms. Gracias a ello, permite emplear validadores tanto personalizados como de Angular, para comprobar que los campos estén rellenos correctamente. El componente invoca al servicio AuthService, para realizar la petición POST al endpoint '/users' para el registro y al endpoint '/token/login' para el acceso. Cuando se obtiene el token, se almacena en el almacenamiento local del navegador mediante el servicio Storage, para su uso en futuras peticiones al backend.

Por el lado del backend, Djoser proporciona un conjunto de endpoints por defecto para la autenticación de usuarios, como resetear contraseña o confirmar el correo electrónico. Una vez que se accede al endpoint, Djoser valida el token JWT y, si es válido, permite el acceso a los recursos protegidos, en caso contrario, devuelve un error 401 Unauthorized (RNF-2).

En la figura 4.5 se muestra un ejemplo de error al intentar acceder a un endpoint sin proporcionar el token JWT.

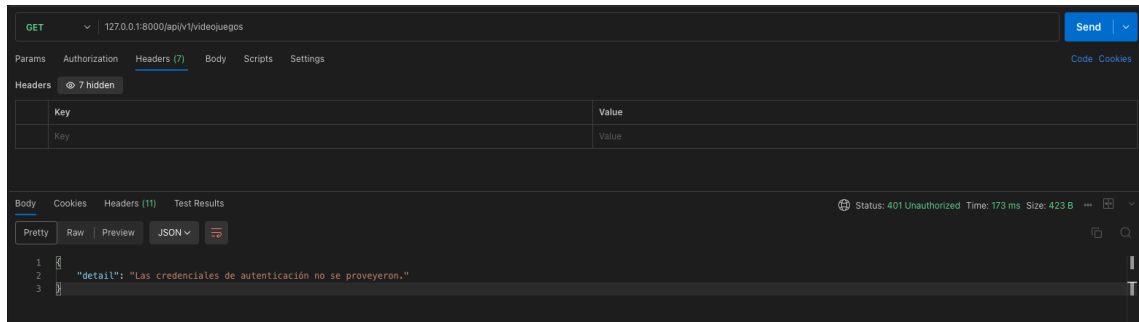


Figura 4.5: Solicitud GET sin token JWT

Mediante PostMan, se realiza un solicitud GET sin incluir ningún token JWT, lo que provoca un error 401 Unauthorized, y el mensaje de error del servidor.

4.3.3. Subsistema de búsqueda

Búsqueda de videojuegos

Desde la pantalla de inicio de la aplicación, se pueden buscar los videojuego por título, género, desarrolladora y/o año de lanzamiento(RF-5.1). En el frontend, la pantalla principal presenta dos componentes. Por un lado, el componente VideogameFilter que presenta un formulario reactivo de búsqueda. En dicho componente, cuando el usuario rellena el formulario y pulsa el botón de búsqueda, se genera un evento mediante EventEmitter, que llama al servicio SearchService, el cual realiza la petición con los parámetros de búsqueda al endpoint 'videojuegos/'. Una vez que se obtiene la respuesta del servidor y se mapea a objetos comprensibles, entra en juego el componente VideogameList, que muestra los videojuegos encontrados en forma de tarjetas.

En el servidor, el viewset se configura con DjangoFilterBackend y SearchFilter, lo que permite realizar búsquedas que coincidan con el título o desarrolladora del videojuego. En la figura 4.6 se muestran los resultados de coincidencias con la desarrolladora 'Nintendo'.

Si no se recibe ningún parámetro de búsqueda, se devuelven todos los videojuegos disponibles. La figura 4.7 muestra la interacción de los componentes.

Detalle de videojuego

Dicho sistema también se presenta en la vista de detalle de cada videojuego(RF-5.2). Dicha pantalla viene enlazada con el subsistema de valoración ya que en ella se pueden ver las valoraciones de los usuarios y realizarlas y con el subsistema de listas ya que se pueden añadir videojuegos a las listas personalizadas. Desde el frontend, el componente VideogameDetailComponent muestra la información del videojuego mediante el objeto Videogame (mapeado anteriormente una vez obtenida la respuesta

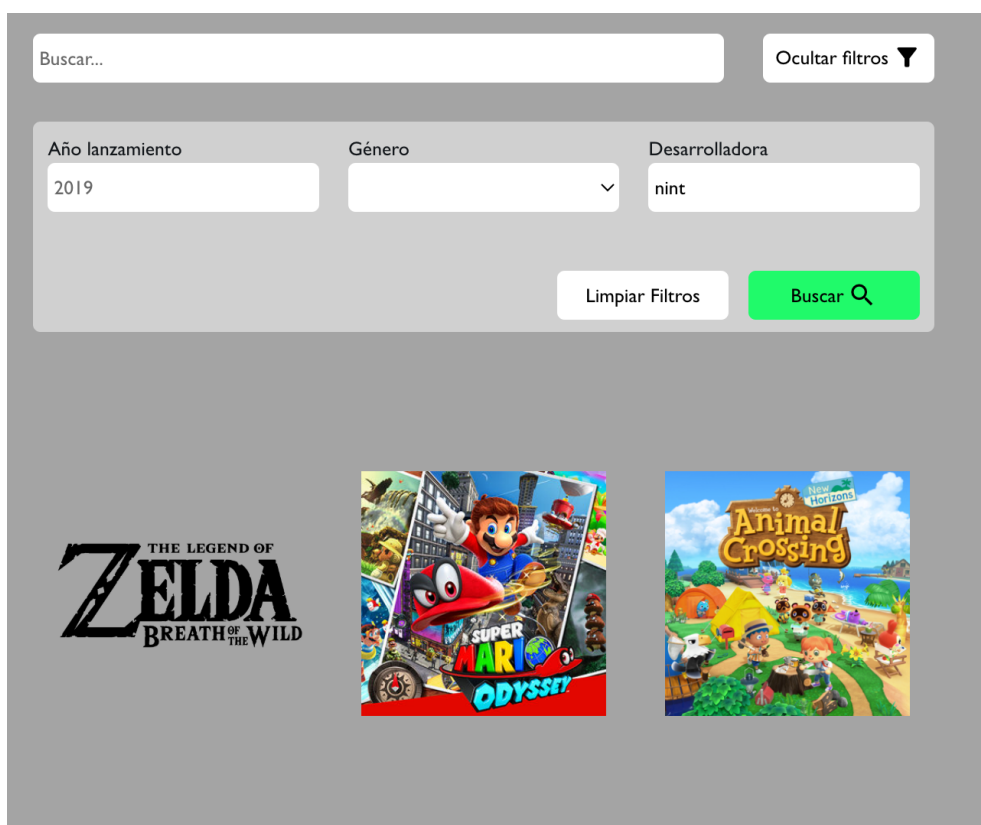


Figura 4.6: Salida de la búsqueda de videojuegos por desarrolladora escribiendo el nombre incompleto

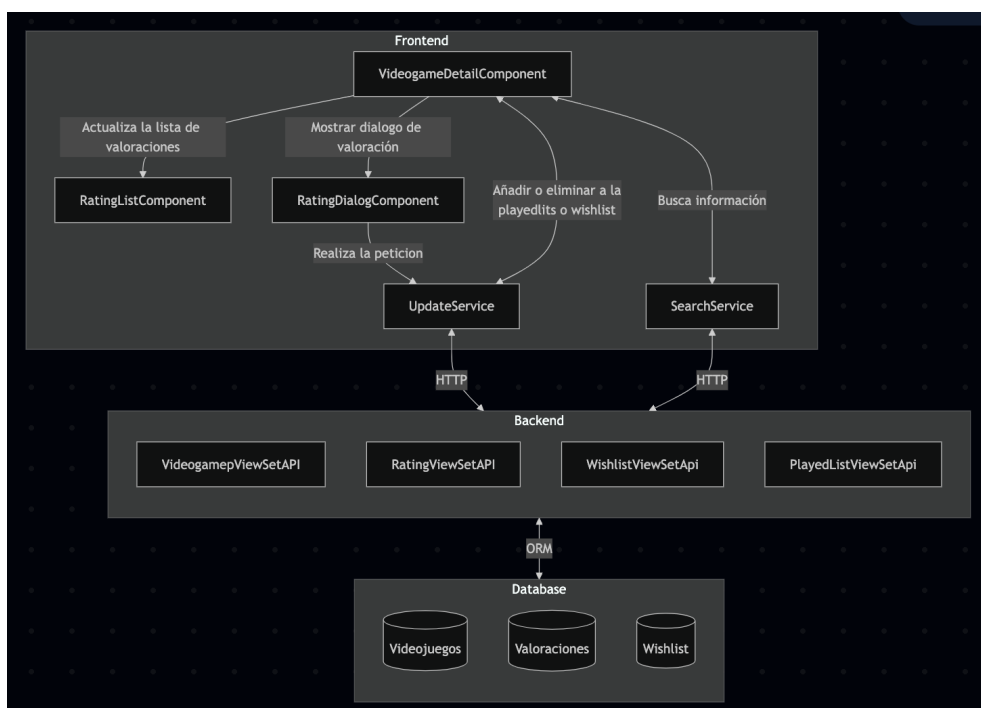


Figura 4.7: En este diagrama se puede ver el proceso completo de la pantalla de detalle de videojuego

del servidor). Dicha vista realiza una petición al servicio SearchService al endpoint 'videojuegos/id', donde id es el identificador del videojuego.

Por el lado del backend, simplemente se obtiene el videojuego con el id indicado en la petición y se devuelve como respuesta.

El flujo de interacción se observa en la figura 4.8.

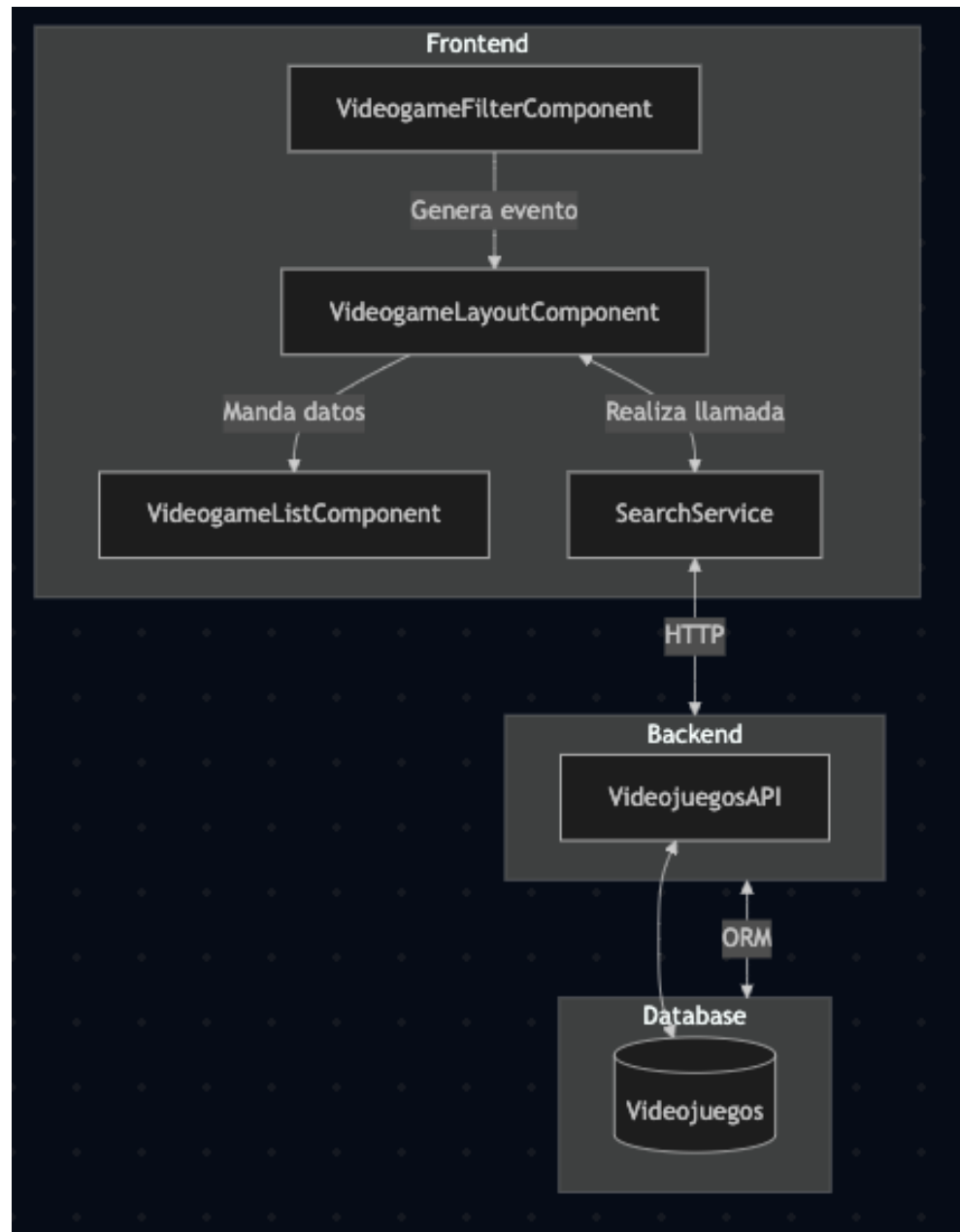


Figura 4.8: En este diagrama se puede ver el proceso completo de la búsqueda de videojuegos

4.3.4. Subsistema de valoración

Detalle de videojuego

En la vista de detalle de cada videojuego, aparece un diálogo implementado en el componente RatingDialog. Dicho componente cuenta con formulario donde se muestran las estrellas de puntuación y un cuadro de texto. Para la realización de las estrellas, se utiliza el componente p-rating de la librería RatingModule. Una vez que el usuario envía el formulario de valoración, se llama al servicio UpdateService, que realiza la petición al endpoint 'ratings/'. Una vez obtenida la respuesta, se actualiza la vista para mostrar las nuevas valoraciones, se actualiza la media de las valoraciones y se muestra un snackbar de confirmación(RF-2). Además cuenta con el componente RatingList que muestra una lista de valoraciones de los usuarios.

En la figura 4.9 se muestra el diálogo de valoración y en 4.10 el mensaje de confirmación.

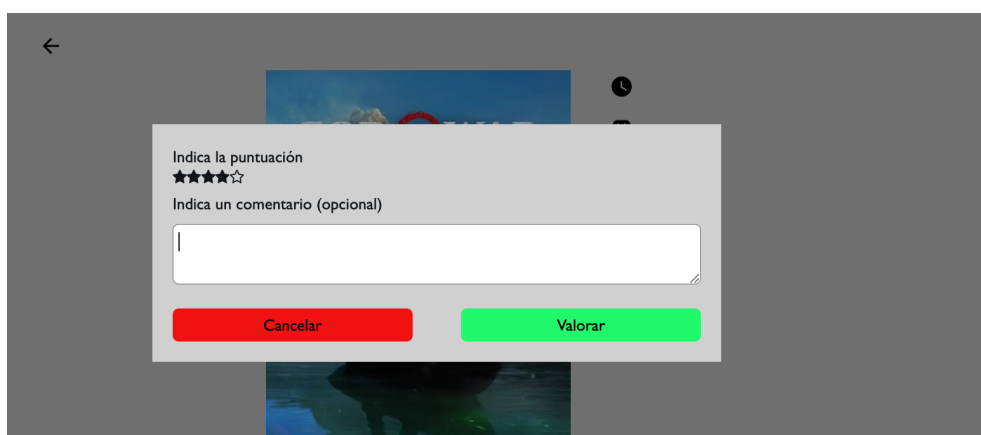


Figura 4.9: Diálogo de valoración de un videojuego

Desde el backend, no se aplica ningún control que limite a un usuario a realizar una única valoración por videojuego, simplemente se añaden más registros a la base de datos. En la figura 4.11 se muestra como el usuario ha realizado varias valoraciones para el mismo videojuego.

Todo el proceso de valoración se observa en la figura 4.7.

4.3.5. Subsistema de listas

Detalle de videojuego

Desde la vista de detalle de un videojuego, el usuario puede añadir o quitar el título de dos listas distintas: la lista de videojuegos jugados (playedList) y la lista de deseos (wishlist). Para añadir el videojuego a la lista, se realiza la petición POST al endpoint 'wishlist/' o 'playedList/'(RF-3.3, RF-3.1). Si el videojuego ya forma parte de una de estas listas, se envía una petición DELETE a wishlist/id o playedList/id, siendo id el identificador del videojuego(RF-3.2, RF-3.4). Cuando se accede a la vista de detalle, el sistema realiza automáticamente una consulta para comprobar si el videojuego ya está

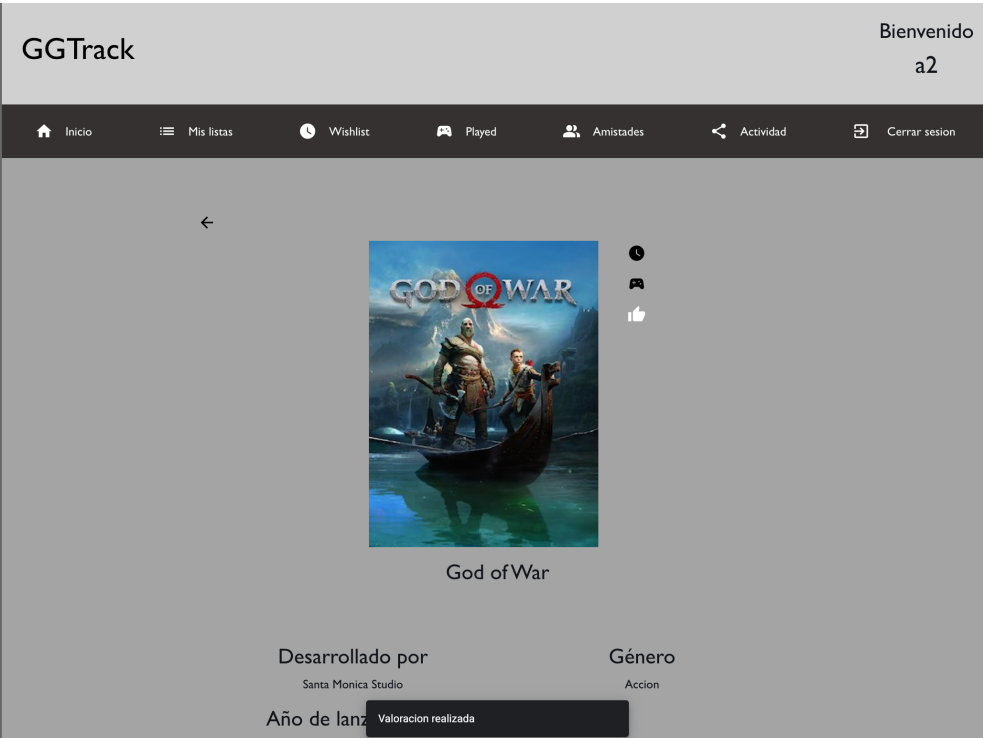


Figura 4.10: Mensaje de confirmación de valoración de un videojuego

id	estrellas	comentario	idUsuario_id	idVideojuego_id
04d17c97-0048-4840-bd06-7608a43d8385	5.0	Esperando al ragknarok	18088e36-2bb2-43e9-b680-3e9364198212	91688bc3-d5ae-4792-828f-6dfbbb662b38
dc748bea-14fa-4370-a106-2b92296b2397	4.0		18088e36-2bb2-43e9-b680-3e9364198212	91688bc3-d5ae-4792-828f-6dfbbb662b38
10e459d1-dde6-4f91-942a-8a875c3c9900	4.0		18088e36-2bb2-43e9-b680-3e9364198212	91688bc3-d5ae-4792-828f-6dfbbb662b38

Figura 4.11: Consulta en base de datos de las valoraciones de un usuario al mismo videojuego

incluido en alguna de las listas. En función de esa comprobación, se modifica el icono del botón (coloreándolo de negro si ya está añadido) y se muestra un tooltip que indica la acción que se realizará al pulsarlo.

En el backend, se devuelve únicamente un indicador de éxito o fallo según el resultado de la operación de añadido o eliminación.

El flujo de interacción se observa en la figura 4.7

Sección 'Mis listas'

Desde la pantalla de listas personalizadas, se ofrecen dos vistas, una para crear la lista y otra para visualizar las listas. A la hora de visualizar las listas, el componente CustomListLayout realiza una solicitud al endpoint 'myLists/'. Una vez que se obtiene la respuesta, este componente, llama a otro componente, encargado de la visualización de las listas. Para cada lista, se pueden realizar dos acciones 'Eliminar' o 'Editar' (RF-3.5, RF-3.6). Para aplicar esas acciones, se realiza la petición PUT o DELETE al endpoint 'api/v1/myLists/<str:pk>', donde pk es el identificador de la lista.

A la hora de crear listas, el usuario puede definir un nombre y una descripción para cada lista personalizada. El formulario de creación de listas realiza una solicitud POST al endpoint 'api/v1/myLists/' enviando los datos proporcionados por el usuario. Antes de enviar los datos, se comprueba desde el front que se ha proporcionado un nombre y videojuegos asociados a esa lista(RF-3.5.1). Por último, en el backend, se valida la información y, si es correcta, se almacena la nueva lista en la base de datos.

El flujo de interacción se observa en la figura 4.12

Sección 'Played' y 'Wishlist'

Las pantallas de 'Played' y 'Wishlist' se implementan de manera similar realizando un petición GET al endpoint '/wishlist' o '/playedList'. La petición la realiza el componente WishlistLayout y para la visualización de los videojuegos se reutiliza el componente VideogameList.

Del lado del backend, en cada view set se obtienen los videojuegos asociados en función del atributo 'IsPlayedList' de la tabla 'Wishlist'.

El flujo de interacción se observa en la figura 4.13

4.3.6. Subsistema de amistad

Sección amistad

Desde la sección de amistad, es posible establecer amistad y confirmar o rechazar solicitudes. El componente padre de esta sección es FriendshipComponent, que contiene dos componentes hijos:

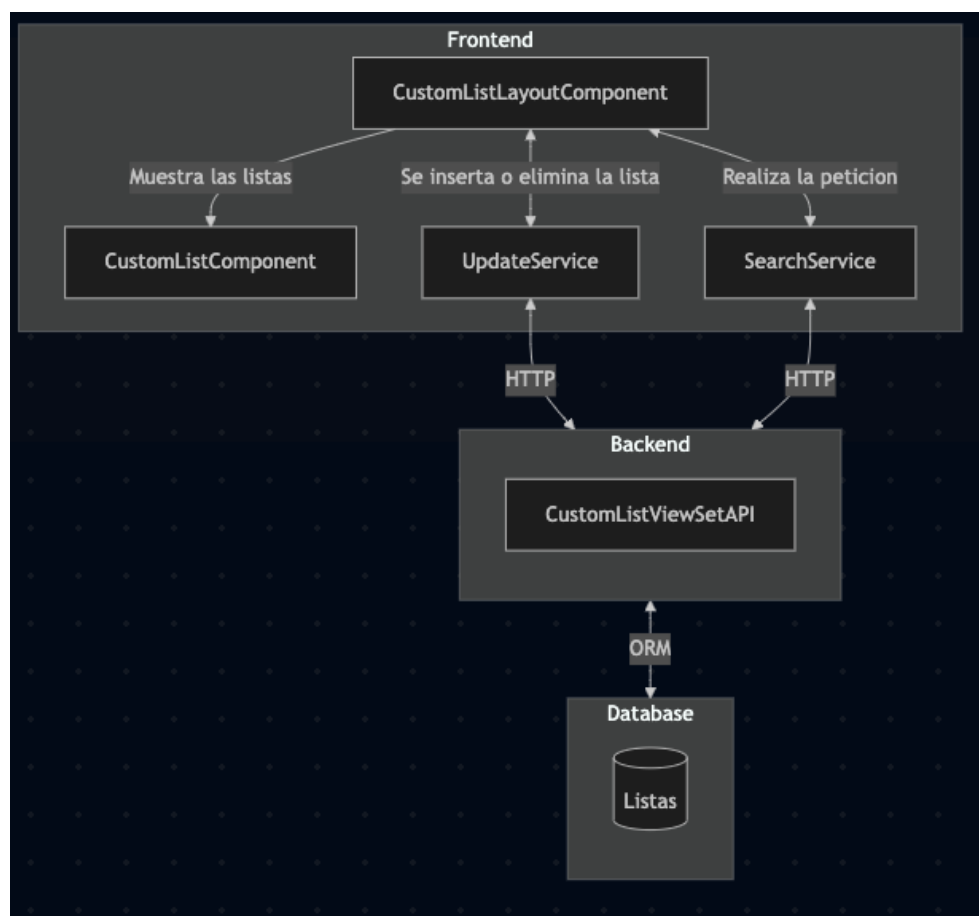


Figura 4.12: En este diagrama se puede ver el proceso completo de la gestión de listas

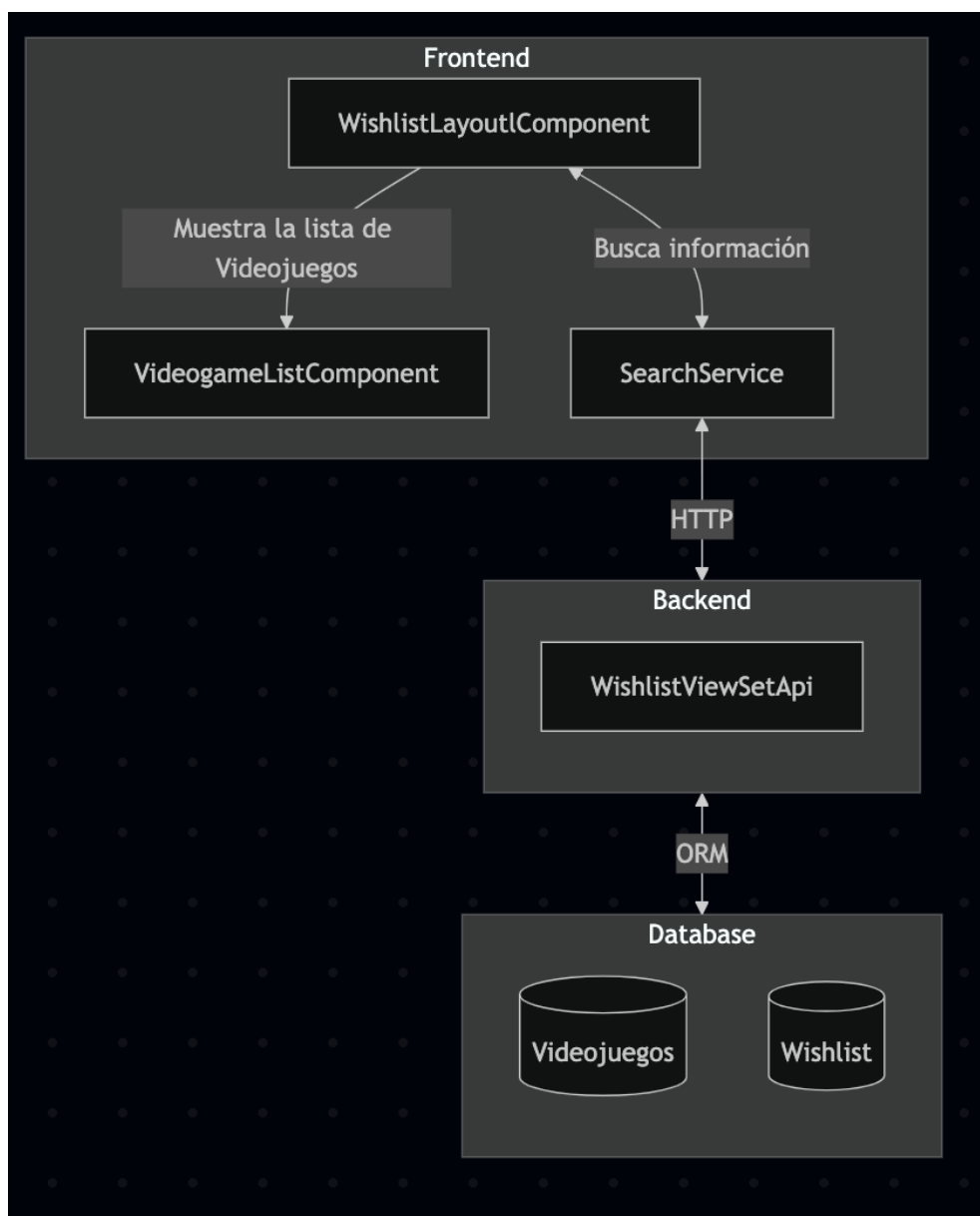


Figura 4.13: En este diagrama se puede ver el proceso completo de la gestión de amistades

UserListComponent y RequestFriendshipComponent. Para iniciar una solicitud es componente UserListComponent es el encargado. El usuario abre un diálogo que incluye un campo de búsqueda para introducir el nombre de otro usuario. Al hacer clic en buscar, se realiza una petición al endpoint allUsers/, que devuelve la lista de usuarios sin privilegios respetando su anonimato. La búsqueda de los usuarios por nombre emplea la misma herramienta de coincidencias que los videojuegos (DjangoFilterBackend y SearchFilter). Los resultados se mapean a objetos Usuario y se muestran mediante el componente UserList, cada uno acompañado de un botón de acción. Al pulsar dicho botón, se envía una solicitud de amistad al endpoint 'friendships/'.

Por el lado del backend, se comprueba si ya existe una solicitud en marcha o si ya son amigos. En caso de que se cumplan esas condiciones, se devuelve un error, de lo contrario, se crea la solicitud de amistad y se establece su estado a 'pendiente'.

Cuando un usuario recibe una solicitud de amistad entra en acción el componente RequestFriendshipComponent, este muestra un recuadro para aceptar o rechazar la solicitud. Si se acepta, se actualiza el estado a "aceptada" mediante una nueva petición al endpoint friendships/. En caso de rechazarla, la solicitud no se elimina, sino que queda registrada como "rechazada". En ambos casos, la interfaz se actualiza automáticamente con los nuevos estados recibidos del backend.

La figura 4.14 muestra el flujo de interacción de la sección de amistad.

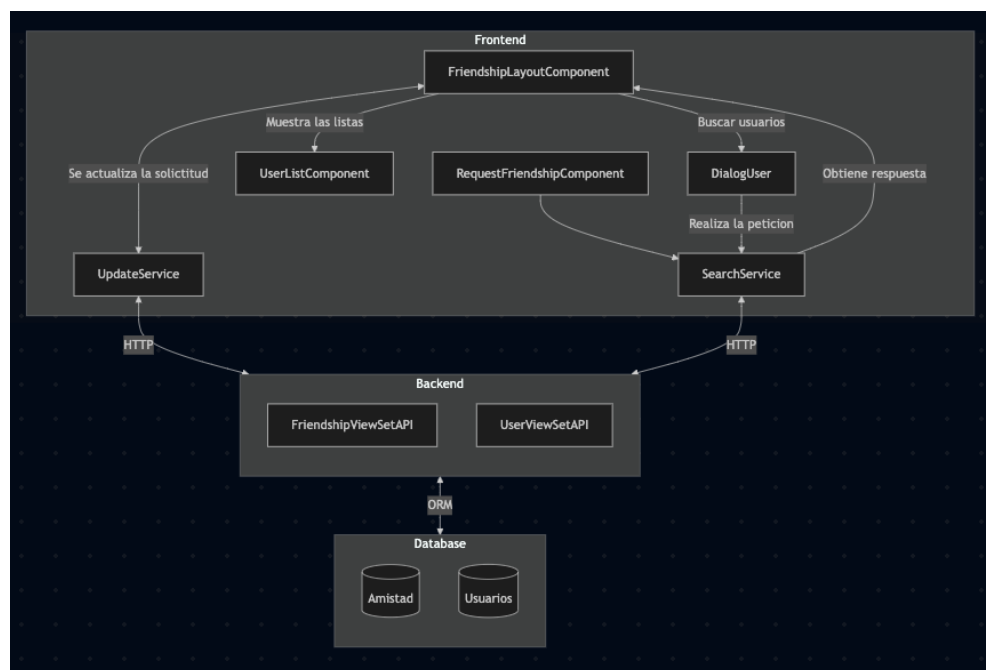


Figura 4.14: En este diagrama se puede ver el proceso completo de la gestión de amistades

Sección actividad

En la pantalla de actividad, se muestra el listado de valoraciones de los usuarios con los que se ha establecido amistad. Para ello, se realizan dos peticiones, una para obtener los usuario amigos y otra para obtener las valoraciones del sistema. Una vez obtenido los resultados, se filtra desde el frontend las valoraciones de los usuarios amigos y se muestran en la pantalla. También se realiza el mismo flujo para obtener los videojuegos jugados o añadidos a la lista de deseos por los amigos.

4.4. Pruebas

Para asegurar el correcto funcionamiento de la aplicación, se han desarrollado pruebas tanto en el backend como en el frontend.

TestCase

Las pruebas del backend se han implementado utilizando la librería TestCase [10]. Se han creado pruebas específicos para cada uno de los módulos principales del sistema:

Módulo de autenticación: se verifican el registro, inicio de sesión y verificación del token.

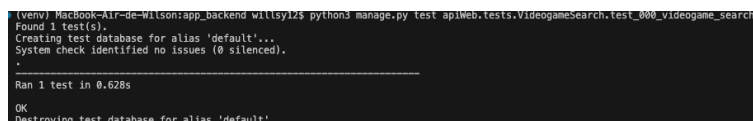
Módulo de amistad: se comprueba el envío, aceptación y eliminación de solicitudes de amistad.

Módulo de búsqueda: se validan las consultas de videojuegos y listas.

Estas pruebas se ejecutan utilizando una base de datos temporal que se crea y destruye para cada prueba, lo que permite no afectar a los datos reales del sistema. Cada prueba se ejecuta siguiendo un patrón:

- 1.– Preparación del entorno: se crea un usuario de prueba y se insertan videojuegos de prueba.
- 2.– Ejecución de la petición: se realiza una petición HTTP al endpoint correspondiente del servidor.
- 3.– Verificación de la respuesta: se comprueba el código de estado HTTP y el contenido de la respuesta utilizando aserciones.

En el código 4.2 se muestra un ejemplo de prueba de búsqueda de videojuegos y en la figura 4.15 se muestra su salida.



```
(venv) MacBook-Air-de-Wilson:app_backend wilson$ python3 manage.py test apiWeb.tests.VideogameSearch.test_000_Videogame_search
Found 1 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.
-----
Ran 1 test in 0.628s
OK
Destroying test database for alias 'default'...
```

Figura 4.15: Salida de una prueba de TestCase que comprueba la búsqueda de videojuegos

Código 4.2: En este código se presenta la ejecución y verificación de la respuesta

```
1 def test_000_videogame_search(self):
2     response = self.client.get(f'{BASE_URL}videojuegos/',
3                               HTTP_AUTHORIZATION=self.auth_header)
4     self.assertEqual(response.status_code, status.HTTP_200_OK)
5     self.assertEqual(len(response.data), 2)
```

Cypress

Para el frontend, se han realizado pruebas end-to-end utilizando la herramienta Cypress [11], que permite simular la interacción del usuario con la interfaz.

Se han desarrollado tres pruebas principales que validan los siguientes aspectos:

Funcionamiento del formulario de login y registro.

Búsqueda de videojuegos desde la barra de búsqueda.

Creación de listas de videojuegos por parte del usuario.

Estas pruebas permiten comprobar que los componentes de la interfaz responden correctamente a interacciones del usuario y que la comunicación con el backend se realiza correctamente.

Cada prueba contiene:

- 1.— Verificación de la interfaz: se comprueba que los elementos de la interfaz existan y que se muestren correctamente.
- 2.— Simulación de la interacción del usuario: se simulan acciones como hacer clic en botones, rellenar formularios y navegar por la aplicación.

En la figura 4.16 se muestra un ejemplo de prueba donde se comprueba que los elementos de la interfaz existen y las respuestas del servidor son correctas y en la figura 4.17 realiza una interacción del usuario.

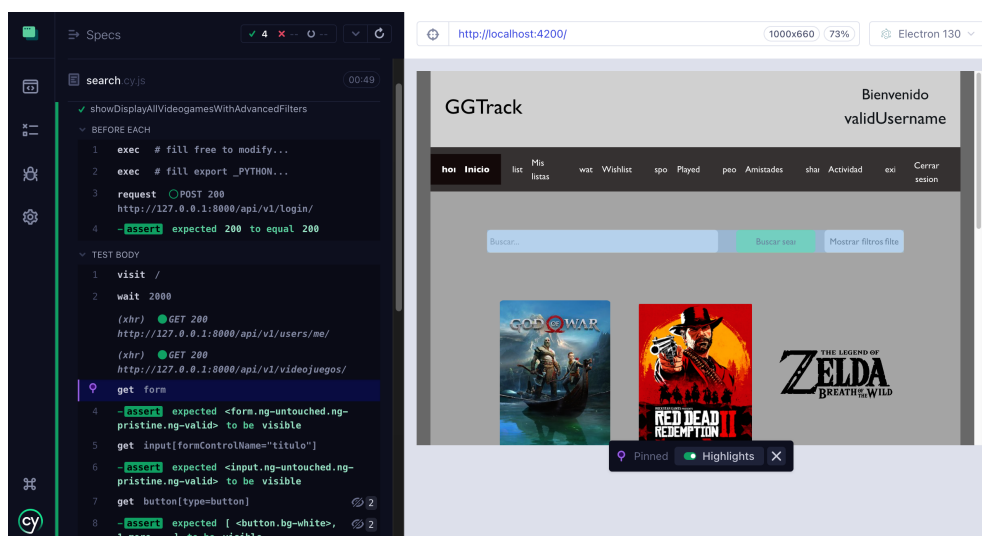


Figura 4.16: Salida de una prueba de Cypress que comprueba elementos de la interfaz y respuestas del servidor

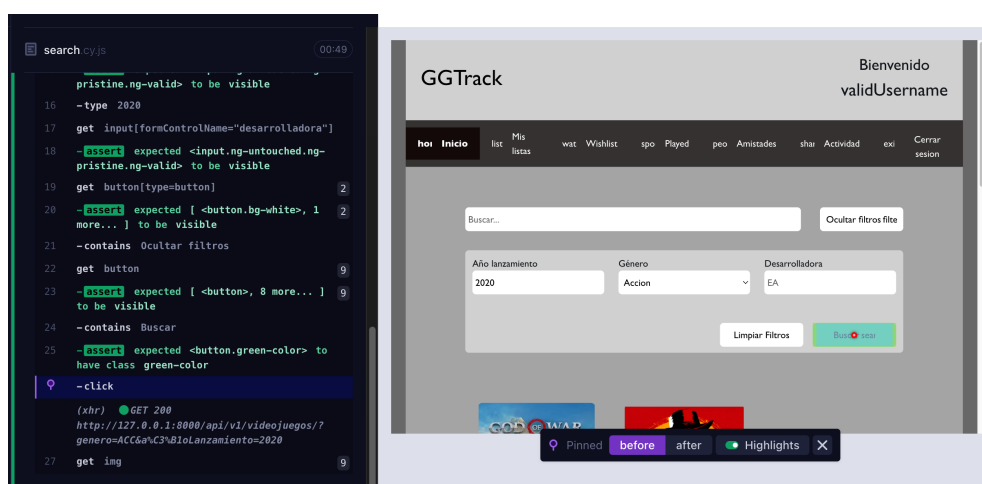


Figura 4.17: Salida de una prueba de Cypress que simula interacciones del usuario pinchando en un botón y rellenando un formulario

CONCLUSIÓN

5.1. Conclusiones

La realización de este TFG supone la creación de una aplicación web que permite a los usuarios gestionar sus títulos de videojuegos favoritos y compartir sus opiniones con otros usuarios.

Con la ausencia de aplicaciones similares, se ha ido desarrollando este proyecto, estableciendo desde el inicio las necesidades y funcionalidades que se querían implementar. Además, se ha realizado un análisis de las tecnologías más adecuadas para su desarrollo, teniendo en cuenta la experiencia previa en los lenguajes de programación.

Por otro lado, la realización de mockups ha permitido definir la interfaz de usuario y la experiencia de usuario, lo que ha facilitado la implementación de la aplicación. La realización de pruebas ha permitido detectar errores y mejorar la usabilidad de la aplicación, asegurando que cumple con los requisitos establecidos.

Además, la utilización de Angular y Django ha permitido una implementación más rápida y eficiente. Angular ha proporcionado una arquitectura modular y herramientas para la creación de componentes reutilizables (como en el caso de la lista de videojuegos). Aunque ha presentado problemas en el desarrollo como el mapeo de los datos de la API a los componentes o la comunicación entre componentes, se ha logrado una interfaz de usuario intuitiva y funcional. Por otro lado, Django ha facilitado la creación de la API, la gestión de la base de datos y la autenticación de usuarios.

En conclusión, la realización de este TFG ha permitido afianzar conocimientos en el desarrollo de aplicaciones web. A nivel personal, ha supuesto un aprendizaje en la estructuración de proyectos, planificación y resolución de problemas y, además, ha resultado especialmente motivador dar vida a una idea pensada desde el principio para los amantes de los videojuegos.

5.2. Trabajo futuro

A continuación, se detallan las mejoras que se consideran necesarias para el futuro.

En primer lugar, se considera necesario la posibilidad de editar valoraciones ya realizadas del usuario por si se ha cometido algún error al introducir la puntuación o el comentario. Para ello, se podría implementar un botón de edición en la lista de valoraciones del usuario, que al pulsarlo, se muestre un diálogo con el formulario de valoración ya rellenado. También, se valora que los usuarios puedan responde a las valoraciones de otros usuarios, para fomentar la interacción entre ellos.

En segundo lugar, se considera la posibilidad de añadir un sistema de idioma para la aplicación, para que el usuario pueda elegir el idioma. Para ello, se podría implementar un selector de idiomas en la barra de navegación que permita al usuario cambiar el idioma.

En tercer lugar, se podría implementar un sistema de edicion de la información del usuario, para que pueda cambiar su nombre de usuario, correo electrónico o contraseña. Incluso se podrían ampliar los campos del usuario para que pueda añadir una foto de perfil o una breve biografía.

En cuarto lugar, Angular ofrece la librería NGRX [12] que permite gestionar el estado de la aplicación de forma más eficiente. De tal forma que los filtros que se aplicaron en una pantalla se mantengan al navegar a otra pantalla y volver a la anterior, sin necesidad de volver a aplicar los filtros.

En quinto lugar, la implementación de un servicio caché permitiría almacenar los datos recibidos del backend y evitar realizar peticiones innecesarias al servidor lo que mejoraría al rendimiento de la aplicación.

Por último, se valora implementar un sistema de notificaciones para que el usuario sepa cuando ha recibido una solicitud de amistad, cuando ha recibido una respuesta de una valoración o para informar de la actividad de sus amigos.

BIBLIOGRAFÍA

- [1] C. de los proyectos Wikimedia, "Igdb - wikipedia, la enciclopedia libre," <https://en.wikipedia.org/wiki/IGDB>, Jun. 2025, consultado en junio de 2025.
- [2] L. Staff, "Front end vs. back end ¿cuál es la diferencia?" <https://hub.laboratoria.la/front-end-vs-back-end-cual-es-la-diferencia>, Nov. 2021, consultado en mayo de 2025.
- [3] C. de los proyectos Wikimedia, "Vue.js - wikipedia, la enciclopedia libre," <https://es.wikipedia.org/wiki/Vue.js>, Oct. 2020, consultado en mayo de 2025.
- [4] —, "React - wikipedia, la enciclopedia libre," <http://es.wikipedia.org/wiki/React>, Aug. 2015, consultado en mayo de 2025.
- [5] —, "Angular (framework) - wikipedia, la enciclopedia libre," [https://es.wikipedia.org/wiki/Angular_\(framework\)](https://es.wikipedia.org/wiki/Angular_(framework)), Mar. 2017, consultado en mayo de 2025.
- [6] —, "Django (framework) - wikipedia, la enciclopedia libre," [https://es.wikipedia.org/wiki/Django_\(framework\)](https://es.wikipedia.org/wiki/Django_(framework)), Dec. 2005, consultado en mayo de 2025.
- [7] —, "Microsoft .net - wikipedia, la enciclopedia libre," https://es.wikipedia.org/wiki/Microsoft_.NET, Apr. 2005, consultado en mayo de 2025.
- [8] IBM, "¿qué es java spring boot? | ibm," <https://www.ibm.com/mx-es/topics/java-spring-boot>, Apr. 2020, consultado en mayo de 2025.
- [9] Arsys, "Arquitectura cliente servidor: Qué es, tipos y ejemplos," <https://www.arsys.es/blog/todo-sobre-la-arquitectura-cliente-servidor>, Oct. 2024, consultado en junio de 2025.
- [10] D. Project, "Writing and running tests | django documentation," <https://docs.djangoproject.com/en/5.2/topics/testing/overview/>, Apr. 2025, consultado en junio de 2025.
- [11] C. Documentation, "End-to-end testing: Your first test with cypress," <https://docs.cypress.io/app/end-to-end-testing/writing-your-first-end-to-end-test>, Jan. 2025, consultado en junio de 2025.
- [12] NgRx Team, "NgRx Docs," <https://ngrx.io/guide/store>, Jun. 2025, consultado en junio de 2025.



Universidad Autónoma
de Madrid